

How to Answer Interview Questions

Java Support II — Kaiser Permanente (Mindtel Global)

Production triage · Java · SQL · Kafka · Splunk · JIRA · STAR behavioral

Read the **30-second** answer first in the interview.

Expand to the **full answer** only if they ask "tell me more" or "walk me through step by step."

1. Master Answer Framework

How Every Technical Answer Should Flow

Interviewers at Support II level are not testing if you memorized docs. They test if you can **run an incident calmly**. Use this structure for every technical question:

1. **Clarify impact** — "First I'd confirm blast radius: which service, environment, and how many members or transactions are affected."
2. **Gather evidence** — Name the tool: Splunk, thread dump, Kafka CLI, SQL read-only query.
3. **Hypothesize** — State 1–2 likely causes based on evidence, not guesses.
4. **Mitigate** — What stabilizes users *now* (scale, rollback, kill blocker, DLT poison message).
5. **Resolve + document** — Root cause fix and JIRA timeline with saved Splunk searches.

30-Second Universal Opener: "I follow Detect, Triage, Mitigate, Resolve, Document. I never restart blindly — I correlate logs, metrics, and recent changes first, then stabilize production, then fix root cause and document everything in JIRA."

Opening: "Tell Me About Yourself"

Walk me through your background.

30 sec: "I'm a Java production support engineer with 4+ years at Amdocs building and triaging Spring Boot microservices. I work daily with SQL data validation, Kafka event pipelines, Splunk log analysis, and JIRA incident coordination — including a 2-month onsite go-live where I resolved 15+ production incidents. I'm looking to bring that hands-on triage experience to Kaiser Permanente's healthcare platform."

Full answer (2 min): Start with role fit, not biography. "I'm a backend engineer who moved into production support because I prefer solving real outages over writing features from scratch. At Amdocs I support billing and integration microservices processing 5M+ transactions a month. My typical day is Splunk investigation, thread dump analysis when CPU spikes, Kafka consumer lag checks, and SQL validation when APIs return wrong data. On the USCC T-Mobile migration I was onsite in Mexico for go-live — I led triage on 15+ incidents, ran UAT cycles with cross-functional teams, and kept JIRA updated every 30 minutes for stakeholders. Before that on Turbo Charging I built Kafka-backed REST APIs and learned to correlate deploy timing with Splunk error spikes. I hold a Java HackerRank certification and I'm comfortable on bridge calls — I focus on clear member impact statements and evidence-based mitigation."

Tip: Say "member impact" even for Amdocs examples — then add "similar to how eligibility delays would affect Kaiser members." This bridges your experience to healthcare.

Why Kaiser / Why This Role?

Why do you want to work at Kaiser Permanente in a Support role?

"Kaiser operates at a scale where production reliability directly affects patient and member care — that's meaningful to me. Support II is where I've done my best work: diagnosing Java services under pressure, not greenfield coding. I want to apply my Splunk, Kafka, and SQL triage skills in a healthcare context where accuracy and HIPAA-aware communication matter. The integrated care model also means cross-team coordination — which I've done during go-live cutovers — and that's the part of the job I excel at."

2. Java Production Support — How to Answer

Q1: A Java microservice in production has high CPU but traffic is normal. How do you investigate?

30 sec: "I'd capture three thread dumps 10 seconds apart and look for many RUNNABLE threads stuck at the same stack frame — that indicates a hot loop, not pool exhaustion. I'd parallel-check Splunk for WARN/ERROR and confirm no recent deploy. Mitigation depends on findings: feature flag off, restart hot pods as temporary relief, engage dev with dumps attached."

Full step-by-step answer:

Step 1 — Triage (2 min): Confirm which pods, which region, when CPU started. Check Grafana or kubectl top pod. Ask: any deploy in last 2 hours? Any batch job?

Step 2 — Evidence: Run jstack or kubectl exec ... jstack 1 three times, 10 seconds apart. If the same method appears in RUNNABLE state across dumps — e.g. PlanMatcher.evaluateRules() — that's your CPU burner. If threads are WAITING on HikariPool — that's DB pool issue, not CPU loop.

Step 3 — Splunk: Search for slow requests and correlationIds. Find a sample request that triggers the hot path. Check if specific input (plan code, member type) triggers it.

Step 4 — Mitigate: Short-term: restart affected pods OR disable feature flag for bad code path. Do NOT restart all pods without evidence — you lose thread dumps.

Step 5 — Document: JIRA with dump files, Splunk saved search URL, timeline. RCA: code fix for infinite loop or rules engine regression.

What NOT to say: "I'd restart the server." That sounds Level I.

Q2: You see OutOfMemoryError: Java heap space in logs. What do you do?

30 sec: "First I check if pods are OOMKilled in kubectl describe. I capture a thread dump before heap dump. I check -Xmx settings and Splunk for payload size spikes or cache growth. Mitigation: temporary restart with approval; resolution: heap analysis for leak or increase heap if legitimate growth."

Full answer: OOM means the JVM ran out of heap. I check: (1) Is it one pod or all? One pod = possible leak in that instance. All pods = config or traffic pattern. (2) Splunk: did message sizes grow? Did a cache clear job fail? (3) jcmd <pid> GC.heap_info for quick view. (4) If recurring: heap dump with -XX:+HeapDumpOnOutOfMemoryError or manual jcmd — analyze in Eclipse MAT or VisualVM for leak suspects. (5) Common causes: unbounded cache, collection never cleared, large HTTP payloads deserialized into memory. I document heap size, GC logs, and leak suspect class in JIRA for dev handoff.

Q3: CannotGetJdbcConnectionException — API returning 500s. Walk me through.

"This means HikariCP (or the connection pool) cannot get a DB connection within timeout. My chain: **Splunk** — when did errors start? Correlate with Kafka consumer surge or batch job. **Actuator** — hikaricp.connections.active vs max pool size. If active = max, pool is exhausted. **Thread dump** — many http-nio threads WAITING on HikariPool.getConnection confirms it. **SQL** — run pg_stat_activity: is a long-running query or blocking session holding connections? **Mitigate:** kill blocking session per DBA runbook; temporarily increase pool only as band-aid. **Root cause:** missing index, N+1 query introduced in last deploy, or batch job competing with OLTP. I never only 'increase pool size' without finding why connections aren't returned."

Q4: How do you read a thread dump under time pressure?

"I scan in fixed order — takes about 3 minutes: (1) Search for deadlock — if found, stop and report immediately. (2) Count BLOCKED threads — same lock object repeated = contention. (3) Look at thread pool names: http-nio, kafka-consumer, pool-N — which pool is exhausted? (4) For CPU issues: find RUNNABLE threads and compare stack tops across dumps. (5) Note TIMED_WAITING on sockets — slow external API. I attach the dump to JIRA and highlight the 5–10 lines that matter — interviewers respect that you don't pretend to read 400 threads line by line."

3. SQL — How to Answer

Q5: Members report wrong health plan in the portal. API looks fine. Where do you start?

30 sec: "I validate data before blaming Java. Read-only SQL: check member's latest eligibility row, compare MAX(updated_at) on feed tables, and FULL OUTER JOIN enrollment source vs eligibility DB for mismatches. Then check Kafka lag — sync may be behind or consumer may have failed silently."

Full answer: Wrong data in UI usually means stale DB, not broken REST layer. I get internal member ID — never SSN in ticket. I run: latest rows from member_eligibility ordered by effective_date. I check eligibility_feed MAX(last_updated_ts) per source — if 2 days stale, ETL or consumer is broken. I FULL OUTER JOIN enrollment system vs eligibility DB on member_id — find rows where plan_code differs. If enrollment has Premium but DB has Basic, sync failed. Splunk: search eligibility-consumer for JsonMappingException or deserialization errors — schema drift can cause silent skip. Kafka: consumer-groups --describe — lag zero but data wrong means consumer processed but failed per-message. Fix: replay from offset after schema fix. I document all queries in JIRA — reproducible evidence.

Q6: API is slow. How do you determine if it's Java or the database?

"I split the latency chain. **Splunk access logs:** which endpoint, p95 latency, when started. **Actuator:** Hikari active connections — if maxed, app waits for DB. **Thread dump:** threads in socket read or JDBC wait. **SQL:** pg_stat_activity for queries running > 30s; blocking PIDs. **EXPLAIN ANALYZE** on the exact query from the stack trace. If plan shows Seq Scan on large table — index issue. If one blocking PID holds 200 connections — lock issue. I say clearly: 'API slowness is a symptom; I trace to the slowest hop — usually DB lock or pool exhaustion in my experience.'"

Q7: Explain how you'd use a JOIN to debug a data issue.

"INNER JOIN when both sides must exist — e.g. claim must have valid provider; if INNER returns zero rows, provider is missing. LEFT JOIN to keep all members even if eligibility is null — finds members with no plan row. LATERAL JOIN for latest eligibility per member without duplicate rows. FULL OUTER JOIN to find mismatches between two source systems — the most powerful support query when two feeds disagree. I'd explain with a concrete example: enrollment feed says HMO Premium, eligibility DB says HMO Basic — FULL OUTER JOIN on member_id surfaces that row immediately."

Kafka — How to Answer

Q8: Consumer lag is growing on a patient eligibility topic. What do you do?

30 sec: "kafka-consumer-groups --describe to see lag per partition. If all partitions lag equally and consumer count equals partition count, it's slow processing — not under-scaling. I check Splunk for SQLTimeout in consumer, thread dump for Hikari wait, and DB blocking. Mitigate: fix blocker, send poison message to DLT, scale only if consumers < partitions."

Full answer: Growing lag = consumers cannot keep up with produce rate. Step 1: CLI describe — lag per partition. All partitions = systemic slow handler. One partition = hot key skew. Step 2: Splunk — rebalance loops? Deserialization errors? Processing time logs. Step 3: Consumer is often DB-bound — each message does INSERT/UPDATE. Check blocking sessions. Step 4: Poison pill — one bad message retries forever, lag grows on that partition. Skip to DLT after N retries. Step 5: Mitigation: don't blindly add consumers beyond partition count — useless. Pause non-critical batch competing for DB locks. Step 6: Healthcare angle: "Lag means members may see stale eligibility — I declare Sev-2 and give member impact in bridge opener." Document offset, lag graph, and root cause in JIRA.

Q9: Explain Kafka architecture for a non-developer on the bridge call.

"Think of Kafka as a secure message tube between systems. Producers drop enrollment events into labeled tubes called topics. Each tube has multiple lanes — partitions — so parallel processing is possible. Consumer services read from their lane and track how far they've read using a bookmark called an offset. Lag is how far behind the reader is. Right now our eligibility reader is 45,000 messages behind — that's why member data is stale. We're fixing the database bottleneck that's slowing the reader."

Q10: What is a poison pill message and how do you handle it?

"A poison pill is a message that crashes the consumer handler every time it's processed — malformed JSON, unexpected null, or bad foreign key. Because Kafka is at-least-once, the consumer retries, never commits offset, and lag grows forever on that partition. I identify it by: lag on one partition only, same exception in Splunk repeated every few seconds, same offset in logs. Mitigation: move message to Dead Letter Topic after 3 retries, skip offset manually per runbook, fix data or schema, replay from offset. Long-term: idempotent consumer with dedup table and schema validation in CI."

4. Kafka, Splunk & JIRA — How to Answer

Q11: How do you use Splunk during a production incident?

30 sec: "I bound time with earliest=-1h, start with ERROR timechart to find when spike began, rex to count top exceptions, then trace correlationId for a failing request. I save the search and paste URL in JIRA so anyone can reproduce my findings."

Full answer — my Splunk workflow:

- (1) **When:** level=ERROR earliest=-2h | timechart span=5m count — pinpoints start time. Correlate with deploy index.
- (2) **What:** rex ... Exception | stats count by exception_class — top exception classes.
- (3) **Where:** stats count by host, service — blast radius.
- (4) **Trace:** Pick correlationId from ERROR row → correlationId="..." | sort _time | table _time host level message — full request path.
- (5) **Specialized:** JDBC errors, Kafka rebalance, p95 latency on access logs.
- (6) **Document:** Saved search → JIRA. HIPAA: never search by SSN; use internal correlation ID only.

Interview demo line: "If you give me index name and sourcetype, I can write the SPL live — but in prod I always bound time first to control cost and noise."

Q12: Write / explain an SPL query to find connection pool errors.

```
index=kp_apps earliest=-2h
("CannotGetJdbcConnectionException" OR "SQLTimeoutException"
OR "HikariPool" OR "Connection is not available")
| stats count by host, message
| sort - count
```

"This searches the app index for the last 2 hours filtering known pool-related strings. stats count by host shows which pods are worst affected. sort - count surfaces the noisiest first. If I see spike at 09:12 on hosts x, y, z — I correlate with Kafka lag or batch job start at 09:10. I'd then pivot to thread dump on top host."

Q13: How would you set up a Splunk alert?

"Saved search on ERROR count in 5-minute window. Threshold: baseline + 3 sigma or fixed > 50 for critical services. Alert action: PagerDuty for on-call, auto-create JIRA Sev-2 with search URL in description. Tuning: avoid paging on single timeout — require 2 consecutive breaches. Review false positives monthly. On bridge I reference alert trigger time as official incident start time."

JIRA, Severity & Team Coordination

Q14: Three P1 tickets land at once — Kafka lag, Java OOM, one wrong report. How do you prioritize?

"I use impact framework: safety, patient/member breadth, duration, workaround. **Kafka lag first** — affects all new eligibility updates, broad member impact, no workaround. I start consumer-groups and Splunk. **OOM second** — I assign a teammate to heap analysis while I drive Kafka; K8s may auto-restart giving brief relief. **Single wrong report third** — isolated, finance can use manual correction short-term; I schedule SQL validation after stabilization. I post prioritization rationale in each JIRA ticket so stakeholders understand sequencing — that's Support II behavior, not silent queue hopping."

Q15: How do you communicate on an incident bridge call?

Opening (declare): "I'm declaring Sev-2 for eligibility-enrollment-api in prod-west. Member impact: approximately 12% of enrollment requests returning 502. Started 09:12 UTC per Splunk alert. I'm leading technical triage."

Update (every 30 min): "Current status: investigating DB pool exhaustion correlated with Kafka lag +45k. Actions taken: thread dump captured, DBA engaged on blocking PID 28491, scaled consumers 4 to 8. Next: monitor lag 30 minutes. Next update 10:00 UTC or sooner if resolved."

Close: "Service stabilized 09:47 UTC. Lag under 1000. Root cause: batch job lock contention. RCA ticket INC-12346 created. No further bridge updates unless regression."

Rules: No PHI on bridge. Use internal service names. Have Splunk URL ready. Don't speculate — say "investigating" if unsure.

Q16: Tell me about leading a production incident. (STAR)

S — Situation: USCC-to-T-Mobile onsite go-live in Guadalajara — 2-month cutover, 2+ microservices, 15+ production incidents in first 48 hours, stakeholders in US and Mexico time zones.

T — Task: Technical triage on integration services — stabilize, debug, keep migration on track for UAT sign-off.

A — Action: Established 30-minute JIRA update rhythm with Splunk links. Prioritized migration blockers over cosmetic defects. Correlated ERROR spikes to specific REST endpoints and SQL failures. Coordinated 3 UAT cycles with cross-functional teams. Escalated blockers to bridge; documented playbooks for offshore handoff.

R — Result: All 15+ incidents resolved within cutover window. Migration sign-off on schedule. Repeat incidents dropped because playbooks were in JIRA. Manager cited communication quality on post-go-live review.

Q17: Tell me about improving incident detection time. (STAR)

S: XL Axiata Jakarta — 8+ microservices, incident detection averaging 45 minutes, fragmented monitoring across teams.

T: Reduce detection time and coordinate upgrade validation across app, infra, and QA.

A: Built unified Splunk dashboard — ERROR rate and dependency health for all 8 services. Weekly JIRA defect triage with clear severity. Created Detect→Triage→Mitigate runbook with named roles. Ran blameless RCAs with owned action items.

R: Detection dropped from 45 minutes to under 5 minutes — 89% reduction. Runbook adopted by adjacent teams.

5. Full Scenario Answers (Say Out Loud)

Scenario A: Kafka Eligibility Lag +45,000

Interviewer: "It's 9 AM Monday. PagerDuty fires: eligibility consumer lag exceeded 45k. Help desk reports stale plan data. Walk me through."

"I'd open with impact and severity. **Sev-2** — not Sev-1 unless authorizations are blocked for urgent care, but 45k lag with stale plan data is broad member impact. I open JIRA INC, join the bridge, confirm topic `patient-eligibility-events`, consumer group `eligibility-consumer-prod`, environment `prod-west`.

First 5 minutes: Splunk — `timechart lag and ERROR count`. I see errors started ~09:00, top exception `SQLTimeoutException`. Kafka CLI — lag spread evenly across all 6 partitions, consumer count is 6 — already at partition limit, so scaling consumers won't help without adding partitions.

Thread dump on consumer pod: `kafka-consumer threads WAITING on HikariPool.getConnection`. SQL read-only: blocking session on `member_eligibility INSERT`, PID 28491, started 08:55 — matches a batch replay job in change log.

Mitigate: DBA kills blocking session per runbook. Pause non-critical batch replay. Monitor lag — should drop over 20–30 minutes.

Communicate: 'Root cause appears batch lock contention with real-time consumer writes. Mitigation in progress. Next update 09:45.'

Post-incident: RCA — separate batch window from real-time consumer path. Add Splunk alert on lag + JDBC errors combined."

Scenario B: Java API CPU 95%, Timeouts

"Four of eight enrollment-api pods at 95% CPU, p95 latency 12 seconds, normal traffic, no recent deploy — I'd rule out traffic surge first.

Three thread dumps 10 seconds apart via `kubectl exec jstack`. Finding: 120 threads `RUNNABLE` at `PlanMatcher.evaluateRules()` — same frame all three dumps. That's an infinite loop or catastrophic rules backtracking, not deadlock or pool wait.

Splunk: trace slow requests — `correlationIds` point to `plan_code MA-PPO-2026-Q3`. Hypothesis: new plan rules config triggers pathological evaluation.

Mitigate: disable feature flag for new rules path if available; drain traffic from hottest pods; restart as temp relief; page dev SME with dumps and sample payload.

I emphasize I did NOT mass-restart without evidence — dumps proved the hot frame. That's the difference between Support I and II."

Scenario C: Wrong Plan — Data Not Code

"47 tickets since 06:00 — systemic, not one member. I get internal ID 7XK-9912. SQL: enrollment source shows HMO Premium, eligibility DB shows HMO Basic. `MAX(last_updated_ts)` on feed B is 2 days old. Kafka lag is zero — consumer is 'caught up' but data is wrong.

Splunk on eligibility-writer: `JsonMappingException` at 05:45 — enrollment service deployed 05:30 with new JSON field consumer doesn't understand. Messages published but never persisted — silent data loss.

Fix: rollback enrollment deploy OR patch consumer schema. Replay Kafka from 05:30 offset. Re-run `FULL OUTER JOIN` — mismatch count should go to zero.

Lesson I state in interview: 'API returned 200 but data was wrong — I validated SQL before opening Java debugger. In healthcare, wrong plan is a compliance and member trust issue, not just a bug ticket.'"

6. Closing Questions, Do's & Don'ts

Questions You Should Ask Them

- "What does the on-call rotation look like for Support II — follow-the-sun or regional?"
- "Which Splunk indexes and sourcetypes do your core Java services log to?"
- "How are Kafka consumer lag alerts configured today — threshold or rate of change?"
- "What's the split between break-fix JIRA incidents and planned Rally work in a typical sprint?"
- "What does success look like in the first 90 days for this role?"

Do	Don't
Lead with member/patient impact	Say "I'd just restart"
Name tools: Splunk, jstack, kafka-consumer-groups	Give LeetCode / algorithm answers
Say "read-only SQL first"	Put PHI in examples (use internal IDs)
Use Detect→Triage→Mitigate→Resolve→Document	Blame another team on bridge
30-sec answer first, expand if asked	Memorize jargon without explaining plain English
Reference your Amdocs go-live, 5M txn, 89% detection improvement	Claim you know Kaiser systems you haven't used

Final 30-Second Close: "I'm a calm, evidence-based production engineer. I triage with Splunk and thread dumps, validate with SQL, monitor Kafka lag, and communicate clearly in JIRA and bridge calls. I've done this at scale during go-live cutovers, and I'm ready to do it for Kaiser members."

Practice method: Read each **30 sec** answer aloud once. Pick 3 **full answers** and record 2-min videos. Day before interview: run Scenario A out loud in under 5 minutes.